

✅ 1. Why Exception Handling Is Important in Web APIs

When something goes wrong inside your API (invalid input, DB failure, null reference, etc.), you should never expose raw exceptions to clients.

A good Web API must:

- Return **consistent error responses**
- Log errors properly
- Hide internal system details
- Convert unhandled exceptions → **proper HTTP status codes**

ASP.NET Core gives you multiple ways to do it:

1. **Try-Catch inside Controller** (small scope, not recommended for global errors)
2. **Exception Filters**
3. **UseExceptionHandler Middleware** (global)
4. **Custom Exception Middleware** (best & recommended)

🧠 2. Global Exception Handling Approaches (Comparison)

Approach	Scope	Recommended?	Notes
Try-Catch in controllers	Local	❌ No	Duplicated code
Exception Filters	Controller/Action level	⚠️ Sometimes	Good for MVC, not for all middleware
UseExceptionHandler	Global pipeline	✅ Yes	Handles all unhandled exceptions
Custom Middleware	Entire pipeline	★ Best	Full control, reusable, logs, custom responses

🔥 3. Hands-On: Creating a Custom Error Response Format

We first define a **common error response model**:

```
public class ErrorResponse
{
    public int StatusCode { get; set; }
    public string Message { get; set; } = string.Empty;
    public string? Details { get; set; } // only for development
}
```

4. Hands-On DEMO: Implementing Custom Exception Middleware

Step 1 — Create Middleware Class

Create a folder **Middleware** → **ExceptionMiddleware.cs**

```
using System.Net;
```

```
using System.Text.Json;
```

```
public class ExceptionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly IHostEnvironment _env;
    private readonly ILogger<ExceptionMiddleware> _logger;

    public ExceptionMiddleware(RequestDelegate next, IHostEnvironment env,
        ILogger<ExceptionMiddleware> logger)
    {
        _next = next;
        _env = env;
        _logger = logger;
    }
}
```

```

public async Task InvokeAsync(HttpContext context)
{
    try
    {
        await _next(context); // execute next middleware
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, ex.Message);
        await HandleExceptionAsync(context, ex);
    }
}

```

```

private Task HandleExceptionAsync(HttpContext context, Exception ex)
{
    var statusCode = (int)HttpStatusCode.InternalServerError;

    var response = new ErrorResponse
    {
        StatusCode = statusCode,
        Message = "An unexpected error occurred!",
        Details = _env.IsDevelopment() ? ex.ToString() : null
    };

    context.Response.ContentType = "application/json";
}

```

```
context.Response.StatusCode = statusCode;

var json = JsonSerializer.Serialize(response);

return context.Response.WriteAsync(json);
}
}
```

Step 2 — Register Middleware in Program.cs

Add this line at the top of the middleware pipeline:

```
var app = builder.Build();

// Global Exception Middleware
app.UseMiddleware<ExceptionMiddleware>();

app.UseAuthorization();
app.MapControllers();
app.Run();
```

5. Hands-On DEMO: Throw an Exception in Controller

Create an API Controller:

```
[ApiController]
[Route("api/[controller]")]
public class DemoController : ControllerBase
{
    [HttpGet("test")]
```

```
public IActionResult Test()
{
    // Force an exception
    throw new Exception("Something broke!");
}
```

6. Output Response (Clean and Consistent)

Call:

GET https://localhost:5001/api/demo/test

You will receive:

```
{
  "statusCode": 500,
  "message": "An unexpected error occurred!",
  "details": "System.Exception: Something broke! ..."
}
```

In Production:

```
{
  "statusCode": 500,
  "message": "An unexpected error occurred!"
}
```

- ✓ No system leak
 - ✓ Clean JSON
 - ✓ Logged internally
 - ✓ Works for all exceptions globally
-

7. Handling Custom Domain Exceptions (Advanced)

Define a custom exception class:

```
public class NotFoundException : Exception
{
    public NotFoundException(string message) : base(message) { }
}
```

Throw it in your service/controller:

```
throw new NotFoundException("Product not found");
```

Modify middleware to detect this:

```
if (ex is NotFoundException)
{
    statusCode = (int)HttpStatusCode.NotFound;
    response.Message = ex.Message;
}
```

8. Using Built-In UseExceptionHandler (Alternative)

If you don't want custom middleware, you can use:

Program.cs:

```
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/error");
}
```

Create ErrorController:

```
[ApiController]
public class ErrorController : ControllerBase
{
    [Route("/error")]
```

```
public IActionResult HandleError() =>
    Problem(title: "An unexpected error occurred");
}
```

❤️ Final Recommendation (Best Practice)

- ✓ Use **Custom Exception Middleware**
- ✓ Create **custom exceptions** for domain/application errors
- ✓ Log everything using **ILogger**
- ✓ Never show stack trace in production